



AUDIT REPORT

April 2026

For



Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	08
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 High Severity Issues	14
1. Operator can override mutually-agreed split percentage at proof time	14
2. OperatorCap has store ability leads to enable unauthorized transfer and freeze	16
 Medium Severity Issues	17
3. record_refund_proof allows bypass of expiry-gated refund flow	17
4. Complete protocol centralization because single OperatorCap controls all operations	18
 Low Severity Issues	19
5. No validation that join_expiry_ms is in the future	19
6. Unbounded payment_proofs vector leads potential storage bloat	20
7. Asymmetric has_joined check in mutual release approval	21
 Informational Issues	22
8. Escrow objects are never destroyed leads permanent on-chain State	22



Automated Tests	23
Closing Summary & Disclaimer	24



Executive Summary

Project Name	Inpact
Protocol Type	Escrow
Project URL	https://www.suiinpact.com
Overview	The sui Inpact contracts implements a milestone-based escrow protocol on Sui, where funds are locked, verified, and released through structured state transitions like Draft → Funded → Resolving → Released. An operator-controlled model (via OperatorCap) governs all critical actions such as funding proofs, developer participation, verification, and settlement, ensuring controlled execution. The escrow supports multiple resolution paths including full release, refund (if developer doesn't join), and mutually agreed split settlements with strict validation checks. A payment proof mechanism records encrypted transaction evidence for funding, release, refund, and split, enabling auditability and off-chain verification. The INPACT_TOKEN is a custom Sui coin (6 decimals) used for payments, with minting controlled via a treasury cap held by the deployer. Together, the contracts form a controlled escrow + token ecosystem, where tokenized payments are managed, verified, and settled through predefined dispute-resolution flows.
Audit Scope	The Scope of the Audit is to analyse the security, correctness and code quality of Inpact Escrow and Token Contracts.
Branch	Main
Contracts in Scope	sources/escrow.move sources/token.move
Commit Hash	7b558e6ad8f131dad8c30307e79955ae280ae867
Language	Move
Blockchain	MoveVM (SUI)



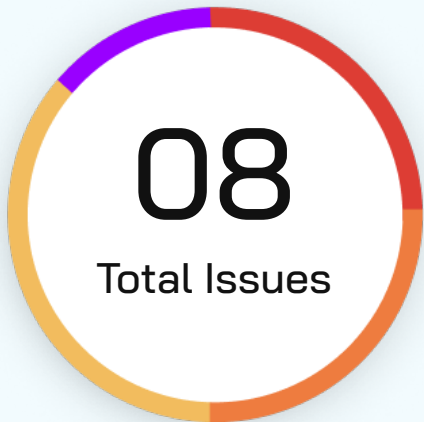
Method	Manual Analysis, Functional Testing, Automated Testing
Source Code link	https://github.com/SuiinpactApp/sui-inpact-core/tree/master/sources
Review 1	27th March 2026 - 30th March 2026
Updated Code Received	1st April 2026
Review 2	1st April 2026
Fixed In	https://github.com/SuiinpactApp/sui-inpact-core/tree/e6bbb2438a055f0384518ecdb29f651de119fede

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0.0%)
High	2 (25.0%)
Medium	2 (25.0%)
Low	3 (37.5.0%)
Informational	1 (12.5%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	1	0	1
	Partially Resolved	0	0	0	0	0
	Resolved	0	2	1	3	0



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Operator can override mutually-agreed split percentage at proof time	High	Resolved
2	OperatorCap has store ability leads to enable unauthorized transfer and freeze	High	Resolved
3	record_refund_proof allows bypass of expiry-gated refund flow	Medium	Resolved
4	Complete protocol centralization because single OperatorCap controls all operations	Medium	Acknowledged
5	No validation that join_expiry_ms is in the future	Low	Resolved
6	Unbounded payment_proofs vector leads potential storage bloat	Low	Resolved
7	Asymmetric has_joined check in mutual release approval	Low	Resolved
8	Escrow objects are never destroyed leads permanent on-chain State	Informational	Acknowledged



Checked Vulnerabilities

✓ Transaction-ordering dependence

✓ Timestamp dependence

✓ Denial of service / logical oversights

✓ Timestamp dependence

✓ Access control

✓ Code clones, functionality duplication

✓ Witness Type

✓ Access Management

✓ Integer overflow/underflow by bit operations

✓ Number of rounding errors

✓ Business logic contradicting the specification

✓ Number of rounding errors

✓ Gas usage

✓ Unchecked CALL Return Values

✓ Centralization of power

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



High Severity Issues

Operator can override mutually-agreed split percentage at proof time

Resolved

Path

sources/escrow.move:L462

Function Name

record_split_proof

Description

When buyer and developer mutually agree on a split via `propose_mutual_split_buyer` and `approve_mutual_split_developer`, the agreed `buyer_share_pct` is stored in `escrow.settlement_buyer_share_pct`. However, `record_split_proof()` accepts a new `buyer_share_pct` parameter and overwrites the agreed value without checking it matches the previously settled percentage.

The function takes `buyer_share_pct` as an input argument, validates only that it is in the 0–100 range via `assert_valid_split_pct`, and then unconditionally writes it into the escrow's settlement field: `escrow.settlement_buyer_share_pct = option::some(buyer_share_pct)`. This completely discards whatever percentage was agreed upon during the mutual negotiation phase.

Vulnerable Code

```
public fun record_split_proof(
    cap: &OperatorCap,
    escrow: &mut Escrow,
    buyer_share_pct: u64,    // Operator can pass ANY value 0-100
    amount: u64,
    ciphertext: vector<u8>,
    clock: &Clock,
    ctx: &TxContext
) {
    assert_operator(cap, ctx);
    assert_valid_split_pct(buyer_share_pct);
    assert!(is_status(escrow, Status::Settled_Split), E_INVALID_STATUS);
    assert!(amount == escrow.amount, E_INVALID_AMOUNT);
    escrow.settlement_buyer_share_pct = option::some(buyer_share_pct);
    // OVERWRITES agreed value without verification
    ...
}
```



Proof of Concept

1. Buyer proposes 50/50 split via `propose_mutual_split_buyer(cap, escrow, 50, ctx)`.
2. Developer agrees via `approve_mutual_split_developer(cap, escrow, 50, ctx)` – state becomes `Settled_Split` with `settlement_buyer_share_pct = Some(50)`.
3. Operator calls `record_split_proof(cap, escrow, 5, ...)` – overwrites to 5/95 split.
4. On-chain record now shows 5% buyer / 95% developer, contradicting the mutual agreement.
5. Off-chain disbursement systems reading the on-chain state will settle at the tampered ratio.

Recommendation

Validate that the split percentage matches the previously agreed value before overwriting. Add an assertion: `assert!(option::is_some(&escrow.settlement_buyer_share_pct) && *option::borrow(&escrow.settlement_buyer_share_pct) == buyer_share_pct, E_MUTUAL_APPROVAL_MISMATCH)`. Remove the line that overwrites the settlement field entirely, as it should already be correctly set by `approve_mutual_split_developer`.

Suggested Fix

```
// FIX: Enforce the agreed-upon split
assert!(
    option::is_some(&escrow.settlement_buyer_share_pct) &&
    *option::borrow(&escrow.settlement_buyer_share_pct) == buyer_share_pct,
    E_MUTUAL_APPROVAL_MISMATCH
);
// Do NOT overwrite escrow.settlement_buyer_share_pct
```



OperatorCap has store ability leads to enable unauthorized transfer and freeze

Resolved

Path

sources/escrow.move:44-46

Function Name

`struct OperatorCap`

Description

OperatorCap is defined with both key and store abilities. The store ability on a capability object is a well-known Sui anti-pattern. It allows anyone who currently holds the OperatorCap to call `transfer::public_transfer(cap, any_address)` – transferring total protocol control to an arbitrary address – without involving the `sui_inpact` module at all.

Additionally, with store, the cap can be frozen via `transfer::public_freeze_object`, permanently bricking all operator-gated functions with no recovery path. The module has no explicit `transfer_operator_cap` function with validation, logging, or multi-sig checks. A single compromised key means instant, silent takeover of all escrow operations.

Vulnerable Code

```
public struct OperatorCap has key, store { // DANGEROUS: store allows
    id: UID,                               // unauthorized transfer/freeze
}
```

Proof of Concept

1. Attacker compromises the operator's private key (phishing, key leak, etc.).
2. Attacker calls `transfer::public_transfer(cap, attacker_address)` in a single transaction.
3. Attacker now holds the OperatorCap and has full control over all escrow state transitions.
4. Alternatively, attacker calls `transfer::public_freeze_object(cap)` to permanently brick the protocol – no escrow can ever transition again.

Recommendation

Remove the store ability from OperatorCap. Implement an explicit, event-emitting transfer function within the module. This ensures transfers are logged and can include additional guards (multi-sig, timelock, etc.).

Suggested Fix

```
public struct OperatorCap has key { // No store
    id: UID,
}

public fun transfer_operator(
    cap: OperatorCap,
    new_operator: address,
    _ctx: &TxContext
) {
    event::emit(OperatorTransferred { new_operator });
    transfer::transfer(cap, new_operator);
}
```



Medium Severity Issues

record_refund_proof allows bypass of expiry-gated refund flow

Resolved

Path

escrow.move:440-460

Function Name

`record_refund_proof()`

Description

The `record_refund_proof` function accepts two statuses: `Refund_Unmatched` and `Partially_Funded`. The `Partially_Funded` branch allows the operator to issue a refund before the join expiry has passed, completely bypassing the intended flow where `refund_unmatched()` first checks `clock::timestamp_ms(clock) > escrow.join_expiry_ms`.

This means the operator can unilaterally cancel any escrow immediately after funding, before giving the developer a fair window to join. The developer may have already prepared work or committed resources based on the escrow's existence, and the premature cancellation violates both parties' expectations about the contract lifecycle.

Vulnerable Code

```
assert!(  
    is_status(escrow, Status::Refund_Unmatched) ||  
    is_status(escrow, Status::Partially_Funded), // Bypasses expiry check  
    E_INVALID_STATUS  
);
```

Proof of Concept

1. Buyer funds escrow via `record_funding_proof` – status becomes `Partially_Funded`.
2. Developer sees the escrow and begins preparing to join (still within `join_expiry_ms` window).
3. Operator calls `record_refund_proof` while status is still `Partially_Funded` – no expiry check is applied.
4. Escrow moves directly to `Released` with a refund proof, denying the developer the agreed-upon join window.

Recommendation

Remove `Partially_Funded` from the accepted statuses in `record_refund_proof`. Refunds from the `Partially_Funded` state should only go through the `refund_unmatched()` function, which correctly enforces the expiry check. If early cancellation is an intentional feature, gate it behind the same expiry check or create a separate explicit cancellation function.

Suggested Fix

```
// Only allow refund from the proper expiry-checked state  
assert!(is_status(escrow, Status::Refund_Unmatched), E_INVALID_STATUS);
```



Complete protocol centralization because single OperatorCap controls all operations

Acknowledged

Path

escrow.move (all public functions)

Function Name

All state-modifying functions

Description

Every state-modifying function in the escrow module requires &OperatorCap. A single OperatorCap is minted in init() and transferred to the deployer. Neither the buyer nor the developer can take any action on-chain – all operations are proxied through the operator.

The buyer cannot fund the escrow directly; the operator records the proof. The developer cannot join directly; the operator triggers it. Neither party can approve a release or split directly. The operator can act against a party's interests (see issue 1). If the operator is compromised, malicious, or unavailable (key loss), all active escrows are stuck or manipulable. This design is closer to a centralized custodian than a decentralized escrow.

Vulnerable Code

```
// Every function follows this pattern:
public fun any_function(
    cap: &OperatorCap, // Single point of control
    escrow: &mut Escrow,
    ctx: &TxContext
) {
    assert_operator(cap, ctx); // No-op check (see M-03)
    ...
}
```

Proof of Concept

1. Operator key is lost or compromised.
2. All in-flight escrows (Draft, Partially_Funded, Fully_Funded, Resolving) become permanently stuck – no party can advance or cancel them.
3. If compromised, attacker can selectively release or refund escrows to favored parties.

Recommendation

Consider a hybrid model where the buyer and developer hold on-chain capability objects (e.g., BuyerCap, DeveloperCap linked to escrow ID) that allow them to directly approve releases, splits, and disputes. The operator role could be limited to dispute arbitration and verification steps only.

Inpact Team's Comment

either of developer or buyer wallet address should not be able to track. That's why we let operator relay all the transactions on behalf of them. We know that looks like pretty centralized.



Low Severity Issues

No validation that join_expiry_ms is in the future

Resolved

Path

escrow.move:195-238

Function Name

`create_draft()`

Description

`create_draft` does not check that `join_expiry_ms > clock::timestamp_ms(clock)`. An operator mistake or malicious intent could create an escrow with an already-expired join window, making it impossible for the developer to ever join. The escrow would be stuck in `Partially_Funded` until the operator manually triggers `refund_unmatched`.

Proof of Concept

1. Operator calls `create_draft` with `join_expiry_ms = 0` (or any past timestamp).
2. Escrow is created but no developer can ever join (expiry already passed).
3. Buyer's funds are locked until operator manually refunds.

Recommendation

Add a future-time assertion in `create_draft`: `assert!(join_expiry_ms > now_ms, E_JOIN_EXPIRED)`.

Suggested Fix

```
assert!(join_expiry_ms > now_ms, E_JOIN_EXPIRED);
```



Unbounded payment_proofs vector leads potential storage bloat

Resolved

Path

escrow.move:165-186

Function Name

record_payment_proof()

Description

The payment_proofs vector grows without bound. While only the operator can append to it, there is no cap on how many proofs can be recorded per escrow. Combined with arbitrary-length ciphertext vectors, a compromised operator could bloat escrow object storage. Gas costs for transactions involving the escrow increase as the vector grows.

Proof of Concept

1. Compromised operator repeatedly calls record_funding_proof or similar functions.
2. Each call appends large ciphertext blobs to payment_proofs.
3. Escrow object grows unbounded, increasing gas costs for all subsequent operations.

Recommendation

Add a maximum proof count constant and enforce it: `const MAX_PROOFS: u64 = 10; assert!(vector::length(&escrow.payment_proofs) < MAX_PROOFS, E_TOO_MANY_PROOFS)`. Also consider capping ciphertext length.

Suggested Fix

```
const MAX_PROOFS: u64 = 10;
assert!(vector::length(&escrow.payment_proofs) < MAX_PROOFS,
E_TOO_MANY_PROOFS);
```



Asymmetric has_joined check in mutual release approval

Resolved

Path

escrow.move:165-186

Function Name

record_payment_proof()

Description

The buyer-side mutual release function, approve_mutual_release_buyer, fails to verify the developer's participation status. In contrast, the developer-side equivalent, approve_mutual_release_developer, correctly implements the assert!(has_joined(escrow), E_ALREADY_JOINED) validation. This procedural asymmetry represents a significant inconsistency in the logic and may facilitate future defects during system maintenance or architectural refactoring.

Vulnerable Code

```
// Buyer side - missing has_joined check
public fun approve_mutual_release_buyer(...) {
    assert_operator(cap, ctx);
    move_to_resolving(escrow);
    escrow.mutual_release_approved_buyer = true;
    // Missing: assert!(has_joined(escrow), E_ALREADY_JOINED);
}

// Developer side - correctly includes check
public fun approve_mutual_release_developer(...) {
    assert_operator(cap, ctx);
    move_to_resolving(escrow);
    assert!(has_joined(escrow), E_ALREADY_JOINED); // Present
    ...
}
```

Proof of Concept

1. Escrow reaches Fully_Funded state.
2. Buyer approval is recorded without verifying developer participation.
3. Inconsistent validation across symmetric functions increases refactoring risk.

Recommendation

Add the same has_joined check to the buyer-side approval function for consistency and defense-in-depth.



Informational Issues

Escrow objects are never destroyed leads permanent on-chain State

Acknowledged

Path

escrow.move (no delete/destructuring)

Description

Once an escrow reaches a terminal state (Released, Refund_Unmatched), the shared object remains on-chain indefinitely. There is no cleanup function to destroy settled escrows. Over time this leads to storage bloat and increased indexing costs. Note that shared objects on Sui currently have limitations around destruction – consider wrapping in a delete PTB pattern if supported.

Recommendation

Add a cleanup function for terminal states that emits a final event and deletes the UID.

Example: public fun destroy_released_escrow(cap: &OperatorCap, escrow: Escrow, ctx: &TxContext) with appropriate status assertions.

Inpact Team's Comment

I agree on what you guys concern but at the current stage, I believe skipping it make us a lot easier to move forward unless it can directly cause any serious problem.



Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of Inpact. We performed our audit according to the procedure described above.

Issues of high , medium , low issues were found. Audit was focused on move based smart contract developed by Inpact team. We strongly recommend conducting multiple audits and participating in bug bounty programs to ensure the highest level of security

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

April 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com